Graph Neural Networks — Midterm

Answer Key

Question 1: (Message Passing) 10 points

(a) [5 pts]

(i) The 4-cycle has edges $1{-}2,\ 2{-}3,\ 3{-}4,\ 4{-}1$, so every node has degree 2, and degree 3 after adding the self-loop.

$$\tilde{\mathbf{A}} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}, \qquad \tilde{\mathbf{D}} = \operatorname{diag}(3,3,3,3), \qquad \hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2} = \tfrac{1}{3}\tilde{\mathbf{A}}.$$

(ii) Row v of $\hat{\mathbf{A}}\mathbf{X}$ is $\frac{1}{3}\big(\mathbf{x}_v + \sum_{u\in\mathcal{N}(v)} \mathbf{x}_u\big)$, giving rows $(1,1),\ (2,2),\ (1,2),\ (2,1)$. With row vectors, $(a,b)\mathbf{W} = (a+b,\ -a+b)$:

$$\hat{\mathbf{A}}\mathbf{X}\mathbf{W} = \begin{pmatrix} 2 & 0 \\ 4 & 0 \\ 3 & 1 \\ 3 & -1 \end{pmatrix} \quad\Longrightarrow\quad \mathbf{H}^{(1)} = \operatorname{ReLU}(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}) = \boxed{\begin{pmatrix} 2 & 0 \\ 4 & 0 \\ 3 & 1 \\ 3 & 0 \end{pmatrix}}.$$

(iii) Node 1: $\operatorname{mean}(\mathbf{x}_2, \mathbf{x}_4) = (0, 1.5)$, $(0,1.5)\mathbf{W} = (1.5, 1.5)$, plus $\mathbf{x}_1 = (3,0)$, so $\mathbf{h}_1^{(1)} = (\mathbf{4.5},\ \mathbf{1.5})$.

Node 4: $\operatorname{mean}(\mathbf{x}_1, \mathbf{x}_3) = (3, 1.5)$, $(3,1.5)\mathbf{W} = (4.5, -1.5)$, plus $\mathbf{x}_4 = (0,0)$, then ReLU gives $\mathbf{h}_4^{(1)} = (\mathbf{4.5},\ \mathbf{0})$.

(b) [5 pts]

Relabel the nodes by $\mathbf{P}$: $\mathbf{A}' = \mathbf{P}\mathbf{A}\mathbf{P}^\top$, $\mathbf{X}' = \mathbf{P}\mathbf{X}$. Since $\mathbf{P}\mathbf{P}^\top = \mathbf{I}$, $\tilde{\mathbf{A}}' = \mathbf{P}(\mathbf{A}+\mathbf{I})\mathbf{P}^\top = \mathbf{P}\tilde{\mathbf{A}}\mathbf{P}^\top$, and the degree vector is only permuted, so $\tilde{\mathbf{D}}'^{-1/2} = \mathbf{P}\tilde{\mathbf{D}}^{-1/2}\mathbf{P}^\top$. Hence $\hat{\mathbf{A}}' = \mathbf{P}\hat{\mathbf{A}}\mathbf{P}^\top$ and

$$f(\mathbf{A}', \mathbf{X}') = \operatorname{ReLU}(\mathbf{P}\hat{\mathbf{A}}\mathbf{P}^\top\mathbf{P}\mathbf{X}\mathbf{W}) = \operatorname{ReLU}(\mathbf{P}\hat{\mathbf{A}}\mathbf{X}\mathbf{W}) = \mathbf{P}\,\operatorname{ReLU}(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}) = \mathbf{P}f(\mathbf{A}, \mathbf{X}),$$

because ReLU acts element-wise and $\mathbf{P}$ only reorders rows.

Why it matters: node indices are arbitrary, so a node's embedding must not depend on how the graph was numbered. With a permutation-invariant readout (sum/mean), graph-level outputs are then invariant.

Question 2: (Expressive Power) 10 points

(a) **[5 pts]**

Start with every node coloured $\circ$; new colours are written as letters.

$t = 1$. Every signature is $(\circ, \{\{\circ, \ldots, \circ\}\})$, so the new colour only encodes the degree: A = degree 1, B = degree 2, C = degree 3.

G node	1	2	3	4	5	H node	a	b	c	d	e
colour	B	B	A	B	C	colour	C	B	B	B	A

Histograms: $\{A{:}1,\ B{:}3,\ C{:}1\}$ for *both* graphs, so there is no separation yet.

$t = 2$.

G node	signature	new	H node	signature	new
1	$(B, \{\{B, C\}\})$	I	a	$(C, \{\{A, B, B\}\})$	J
2	$(B, \{\{B, C\}\})$	I	b	$(B, \{\{B, C\}\})$	I
3	$(A, \{\{B\}\})$	D	c	$(B, \{\{B, B\}\})$	L
4	$(B, \{\{A, C\}\})$	F	d	$(B, \{\{B, C\}\})$	I
5	$(C, \{\{B, B, B\}\})$	K	e	$(A, \{\{C\}\})$	E

Histograms: $G = \{I{:}2,\ D{:}1,\ F{:}1,\ K{:}1\}$ and $H = \{I{:}2,\ J{:}1,\ L{:}1,\ E{:}1\}$. They **differ**, so 1-WL separates G and H at iteration $\mathbf{t = 2}$. For example, G's degree-3 node has three degree-2 neighbours $(C, \{\{B, B, B\}\})$, while H's degree-3 node a has a degree-1 neighbour $(C, \{\{A, B, B\}\})$. Likewise, G's leaf hangs off a degree-2 node and H's leaf hangs off a degree-3 node.

(b) **[5 pts]**

Mean GNN: no. With identical inputs, every message is the same vector $\mathbf{h}$, and the mean of any number of copies of $\mathbf{h}$ is again $\mathbf{h}$. Every node in both graphs therefore keeps the same embedding at every layer (mean is blind to degree), and any readout gives the same graph embedding for G and H (both have 5 nodes).

GIN: yes. Xu et al. show that any injective multiset function on a countable domain can be written as $\Phi\big(\sum_{x \in S} f(x)\big)$, and MLPs can model f and Φ. Sum aggregation therefore keeps neighbour *counts*, and the $(1 + \epsilon)$ term weights the node's own feature differently from its neighbours', so (node, neighbour multiset) is encoded injectively. A 2-layer GIN thus reproduces two rounds of 1-WL, and an injective sum readout over the different $t = 2$ colour histograms separates the graphs.

Difference. Mean only sees the *proportions* of neighbour features; sum sees the full multiset (including degree). GIN is as powerful as 1-WL, a mean-GNN is strictly weaker.

Question 3: (Expressive Power) 5 points

(a) [2 pts]

Original graphs. G_1 (two triangles) and G_2 (a 6-cycle) are both 2-regular with 6 nodes, so 1-WL gives every node the same colour at every iteration and cannot separate them.

Node-deleted subgraphs. Deleting one node gives

	subgraph	degrees of the remaining nodes
$G_1 - v$	triangle + one separate edge	$2, 2, 2, 1, 1$
$G_2 - v$	path on 5 nodes	$1, 2, 2, 2, 1$

The degree multisets are equal (so the $t = 1$ histograms agree), but at $t = 2$: $G_1 - v$ has $3 \times (2, \{\{2, 2\}\})$ and $2 \times (1, \{\{1\}\})$, while $G_2 - v$ has $2 \times (1, \{\{2\}\})$, $2 \times (2, \{\{1, 2\}\})$ and $1 \times (2, \{\{2, 2\}\})$. So **yes**, 1-WL tells the two subgraphs apart.

Why ESAN separates the graphs. Both graphs are vertex-transitive, so *every* node-deleted subgraph of G_1 is "triangle + edge" and every one of G_2 is a 5-node path. The bags are $\{\{6 \times (K_3 + K_2)\}\}$ and $\{\{6 \times P_5\}\}$. With an injective GNN on each subgraph and an injective multiset pooling, the two bag embeddings differ. Deleting a node breaks the regularity that made the original graphs look identical to 1-WL.

(b) [3 pts]

(i) Structural features and node identifiers. *GSN* adds counts of predefined substructures (triangles, cycles, cliques) to the node features, so the GNN can distinguish structures beyond the 1-WL limit.

Advantage: Adds meaningful structural information (e.g. molecular rings) without changing the underlying GNN.

Limitation: Substructures must be chosen in advance, and counting them can be expensive.

Random features or node IDs give otherwise symmetric nodes unique identifiers.

Advantage: Simple to implement and increases expressive power.

Limitation: Fixed node IDs break permutation invariance; random identifiers add stochasticity and may need averaging for stable predictions.

(ii) Subgraph GNNs (ESAN). ESAN builds a bag of subgraphs (e.g. node-deleted subgraphs), processes each with a shared GNN, and aggregates the results. This captures structural differences that standard message passing misses.

Advantage: Some subgraph policies are strictly more expressive than 1-WL while reusing standard GNN encoders.

Limitation: Compute and memory grow with the number and size of subgraphs, e.g. $O(n)$ subgraphs for an n-node graph.

(iii) Higher-order GNNs (k-GNNs). Messages are passed between sets or tuples of k nodes instead of single nodes, giving richer structural interactions.

Advantage: Can exceed 1-WL, with theoretical links to the higher-dimensional Weisfeiler–Leman tests.

Limitation: The number of k-node groups grows as $O(n^k)$, which is too expensive on large graphs.

Question 4: (Scalability) 10 points

(a) **[5 pts]**

With full expansion, one target node needs $1 + \bar{d} + \cdots + \bar{d}^L$ embeddings. For $L = 2$, $\bar{d} = 30$:

$$1 + 30 + 30^2 = \mathbf{931} \text{ per target}, \qquad 32 \times 931 = \mathbf{29{,}792} \text{ for the batch.}$$

(b) **[5 pts]**

Full-graph training on `ogbn-papers100M` faces two obstacles:

1. **Memory.** Full-batch training must store node features, intermediate representations and gradients for all 111 M nodes, plus the 1.6 B-edge structure. Memory grows with graph size, hidden dimension and depth, and quickly exceeds GPU capacity.
2. **Gradient updates.** Full-batch gradient descent must process the entire graph before every parameter update. Each update is very expensive and only one update is made per epoch, so training is slow.

Why hub ("celebrity") nodes are especially harmful. Hubs have exceptionally many neighbours:

- **Neighbourhood explosion:** a hub inside a multi-hop neighbourhood sharply increases the number of nodes and edges to process, raising memory and compute.
- **Redundant computation:** hubs appear in the receptive fields of many target nodes, so their representations are accessed or recomputed again and again across mini-batches.

Question 5: (Scalability) 10 points

(a) **[5 pts]**

Δ holds the **2** cross edges $(3, 6)$ and $(4, 5)$.

- $q = 1$: $2/12 = \mathbf{16.7}\%$ of the edges are lost.
- $q = 2$: both clusters are in one batch, so the induced subgraph keeps all 12 edges and **0%** is lost.

In general, putting several random clusters in one batch restores the between-cluster edges among the chosen clusters and balances the label distribution across batches.

(b) **[5 pts]**

Cluster-GCN partitions the graph into small, densely connected clusters and uses them as mini-batches:

1. **Lower memory:** only a subset of nodes and edges is processed at a time, and there is no exponential neighbourhood expansion as in multi-layer neighbour sampling.
2. **Better efficiency:** nodes in a cluster are densely connected, so message passing stays inside the batch and redundant computation is reduced.

Limitations. Inter-cluster edges are dropped from each mini-batch, which restricts information exchange between clusters. Neighbourhood aggregation becomes incomplete, structural information is lost, and accuracy can drop. Clusters may also have skewed label distributions, which biases the stochastic gradients.

Question 6: (Deeper GNNs) 10 points

(a) **[5 pts]**

(i) On the path $1 - 2 - 3$:

$$\text{Layer 1:} \quad h_1^{(1)} = \tfrac{2+4}{2} = 3, \quad h_2^{(1)} = \tfrac{2+4+6}{3} = 4, \quad h_3^{(1)} = \tfrac{4+6}{2} = 5,$$
$$\text{Layer 2:} \quad h_1^{(2)} = \tfrac{3+4}{2} = \mathbf{3.5}, \quad h_2^{(2)} = \tfrac{3+4+5}{3} = \mathbf{4}, \quad h_3^{(2)} = \tfrac{4+5}{2} = \mathbf{4.5}.$$

The difference between neighbouring nodes shrinks from 2 (input) to 1 (layer 1) to 0.5 (layer 2), and the range from 4 to 2 to 1. The representations become progressively more similar, which illustrates **over-smoothing**.

(ii) The number of nodes at distance k from the root is 3^k:

Distance (hops)	0	1	2	3	4
Nodes	1	3	9	27	81

$$N = 1 + 3 + 9 + 27 + 81 = \boxed{121}, \qquad \text{in general} \quad N(L) = \sum_{k=0}^{L} 3^k = \frac{3^{L+1} - 1}{2} = O(3^L).$$

Explanation. The receptive field grows exponentially with depth, but the root's representation stays 4-dimensional. Information from exponentially many nodes must be compressed into a fixed-size vector. This bottleneck causes *over-squashing*, especially when the prediction depends on many distant nodes: their signals get squashed and have little influence on h_{root}.

(iii) A K-layer GNN at a node only sees nodes within K hops. With 2 layers, node 1 sees **nodes 1, 2 and 3**. Node 5 is at distance 4, so at least **4 layers** are needed. With fewer layers, $\partial h_1^{(K)} / \partial x_5 = 0$ exactly: this is *under-reaching*.

(b) **[5 pts]**

Averaging replaces each value by something between itself and its neighbours, so differences between neighbours shrink. Large differences between neighbours are the "high-frequency" part of a graph signal (large Laplacian eigenvalues, large Dirichlet energy $E = \sum_{(u,v)\in\mathcal{E}} (x_u - x_v)^2$). Averaging damps them and keeps the smooth, low-frequency part, so E falls with every step, exactly like a low-pass filter. If $E \to 0$, all neighbouring embeddings are identical, and the classifier has nothing left to separate the classes with.

Question 7: (Deeper GNNs) 10 points

(a) **[5 pts]**

(i) $\frac{2}{3}(3,3,0,3) + \frac{1}{3}(9,0,0,0) = (2,2,0,2) + (3,0,0,0) = (\mathbf{5},\mathbf{2},\mathbf{0},\mathbf{2})$.

(ii) Factor $= (1 - \frac{1}{2}) + \frac{1}{2} \cdot 3 = \mathbf{2}$, so $\mathbf{H}^{(1)} = \text{ReLU}\left(2 \cdot (5,2,0,2)\right) = (\mathbf{10},\mathbf{4},\mathbf{0},\mathbf{4})$.

(b) **[5 pts]**

Initial residual. Every layer keeps a fixed fraction α of the original, never-smoothed features $\mathbf{H}^{(0)}$. Unrolling the layers gives roughly $\sum_k \alpha(1-\alpha)^k \tilde{\mathbf{P}}^k \mathbf{H}^{(0)}$, a personalised-PageRank mixture that stays centred on each node's own input instead of converging to the stationary state (one common vector). This is why repeatedly re-injecting $\mathbf{H}^{(0)}$ preserves node-specific information. A plain residual adds $\mathbf{H}^{(l)}$, which after many layers is itself already smoothed, so it only feeds back blurred information and does not stop the collapse.

Identity mapping. $(1-\beta)\mathbf{I} + \beta\mathbf{W}^{(l)}$ keeps each layer's transformation close to the identity (in GCNII β_l decreases with depth). A deep model is then at least as good as a shallow one, the weights stay well-conditioned, and gradients flow through many layers, which makes deep GCNs easier to optimise.

Question 8: (Heterophily) 10 points

(a) [5 pts]

Class A $= \{1,2,3\}$ (shaded), class B $= \{4,5,6\}$ in both graphs.

G_1. 7 edges, split by the classes of their endpoints:

Edge type	Edges	Count
A–A	$(1,2),\ (1,3),\ (2,3)$	3
B–B	$(4,5),\ (4,6),\ (5,6)$	3
A–B	$(3,4)$	1

- $h_{edge} = (3+3)/7 = 6/7 \approx \mathbf{0.857}$.
- Per node: $1{:}\,1,\ 2{:}\,1,\ 3{:}\,\frac{2}{3},\ 4{:}\,\frac{2}{3},\ 5{:}\,1,\ 6{:}\,1$, so $h_{node} = \frac{1}{6} \cdot \frac{16}{3} = \frac{8}{9} \approx \mathbf{0.889}$.

G_2. 7 edges, split by the classes of their endpoints:

Edge type	Edges	Count
A–A	—	0
B–B	$(4,5)$	1
A–B	$(1,4),\ (1,5),\ (2,5),\ (2,6),\ (3,4),\ (3,6)$	6

- $h_{edge} = (0+1)/7 = 1/7 \approx \mathbf{0.143}$.
- Per node: $1{:}\,0,\ 2{:}\,0,\ 3{:}\,0,\ 4{:}\,\frac{1}{3},\ 5{:}\,\frac{1}{3},\ 6{:}\,0$, so $h_{node} = \frac{1}{6} \cdot \frac{2}{3} = \frac{1}{9} \approx \mathbf{0.111}$.

G_1 is strongly **homophilous**; G_2 is strongly **heterophilous**.

(b) [5 pts]

The four metrics disagree because they **weight different things**, and only some of them **remove the chance level**.

Edge homophily counts edges: every edge has the same weight, so high-degree nodes (hubs, dense groups) dominate. There is no chance correction.

Node homophily computes a ratio per node and then averages over nodes: every node has the same weight, so many low-degree nodes can outvote a few hubs. Example: a 10-node class-A clique (45 same-class edges) plus 20 separate A–B pairs gives $h_{edge} = 45/65 \approx 0.69$ but $h_{node} = 10/50 = 0.2$. There is no chance correction either.

Class homophily computes homophily per class, subtracts that class's share of nodes (what random links would give), clips at 0, and averages over *classes*. Small classes count as much as large ones, and linking to the majority class "by luck" earns nothing.

Adjusted homophily subtracts the homophily expected if edges were rewired at random with the same degrees ($\sum_k p_k^2$) and rescales. $h_{adj} = 0$ means "no better than chance", 1 means perfect homophily, and negative values mean heterophily.

Question 9: (Heterophily) 10 points

(a) **[5 pts]**

Edges: $F_1N_1,\ F_1N_2,\ F_2N_2,\ F_2N_3,\ N_3N_4$; $x = 1$ for fraudsters, $x = 0$ otherwise.

v	x_v	$\mathcal{N}_1(v)$	$\mathcal{N}_2(v)$	$\mathbf{r}_v$
F_1	1	N_1, N_2	F_2	$[1 \,\|\|\, 0 \,\|\|\, 1]$
F_2	1	N_2, N_3	F_1, N_4	$[1 \,\|\|\, 0 \,\|\|\, \frac{1}{2}]$
N_1	0	F_1	N_2	$[0 \,\|\|\, 1 \,\|\|\, 0]$
N_2	0	F_1, F_2	N_1, N_3	$[0 \,\|\|\, 1 \,\|\|\, 0]$
N_3	0	F_2, N_4	N_2	$[0 \,\|\|\, \frac{1}{2} \,\|\|\, 0]$
N_4	0	N_3	F_2	$[0 \,\|\|\, 0 \,\|\|\, 1]$

(b) **[5 pts]**

D1 Ego–neighbour separation. Under heterophily the neighbours look different from the node, so mixing them in (self-loop averaging) erases the node's own evidence. Keeping x_v in its own channel preserves it.

D2 Higher-order neighbourhoods. 2-hop neighbours are often of the same class (e.g. two fraudsters sharing a victim: F_1's 2-hop neighbour is F_2), so they add homophilous evidence that the 1-hop neighbourhood lacks.

D3 Combining intermediate representations. Concatenating the outputs of all layers lets the classifier use both local and wider views, and mixes low- and high-frequency information instead of keeping only the most smoothed one.

Question 10: (Attentive GNNs) 10 points

(a) [5 pts]

Step 1: transform. $\mathbf{W}$ keeps the first feature and doubles the second: $\mathbf{W}\mathbf{h}_1 = (1,0)$, $\mathbf{W}\mathbf{h}_2 = (0,2)$, $\mathbf{W}\mathbf{h}_3 = (1,6)$, $\mathbf{W}\mathbf{h}_4 = (3,4)$.

Step 2: scores. Split $\mathbf{a} = [\mathbf{a}_1 \| \mathbf{a}_2]$ with $\mathbf{a}_1 = (1.4, 0)$ and $\mathbf{a}_2 = (-0.7, -0.7)$. The query part is $\mathbf{a}_1^\top \mathbf{W}\mathbf{h}_1 = 1.4$, and the key part is $\mathbf{a}_2^\top \mathbf{W}\mathbf{h}_j = -0.7 \times$ (sum of the entries of $\mathbf{W}\mathbf{h}_j$).

j	$\mathbf{a}_2^\top \mathbf{W}\mathbf{h}_j$	raw score	e_{1j} (LeakyReLU)	$\exp(e_{1j})$	α_{1j}
1	-0.7	0.7	0.7	2	$\mathbf{\frac{1}{2}}$
2	-1.4	0	0	1	$\mathbf{\frac{1}{4}}$
3	-4.9	-3.5	$0.2 \times (-3.5) = -0.7$	$\frac{1}{2}$	$\mathbf{\frac{1}{8}}$
4	-4.9	-3.5	-0.7	$\frac{1}{2}$	$\mathbf{\frac{1}{8}}$

Step 3: softmax. The sum of exponentials is $2 + 1 + \frac{1}{2} + \frac{1}{2} = 4$, so the weights are $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$ (they sum to 1).

Step 4: aggregate.

$$\sum_j \alpha_{1j} \mathbf{W}\mathbf{h}_j = \tfrac{1}{2}(1,0) + \tfrac{1}{4}(0,2) + \tfrac{1}{8}(1,6) + \tfrac{1}{8}(3,4) = (1, \tfrac{7}{4}), \qquad \mathbf{h}_1' = \mathrm{ReLU}(1, \tfrac{7}{4}) = \boxed{(1,\ \tfrac{7}{4})}.$$

Comparison with uniform (GCN-style) averaging. $\frac{1}{4}\big((1,0) + (0,2) + (1,6) + (3,4)\big) = (\frac{5}{4},\ 3)$. GAT trusts node 1 itself *more* ($\frac{1}{2}$ vs. $\frac{1}{4}$), node 2 equally ($\frac{1}{4}$), and nodes 3 and 4 *less* ($\frac{1}{8}$ vs. $\frac{1}{4}$). Its output stays close to node 1's own features, while uniform averaging is pulled toward the large features of nodes 3 and 4. LeakyReLU changed the two negative raw scores from -3.5 to -0.7; with plain ReLU both would become 0 and get the same weight as node 2.

(b) [5 pts]

Static attention in GAT. Splitting $\mathbf{a} = [\mathbf{a}_1 \| \mathbf{a}_2]$,

$$e_{vu} = \mathrm{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]\right) = \mathrm{LeakyReLU}\left(\mathbf{a}_1^\top \mathbf{W}\mathbf{h}_v + \mathbf{a}_2^\top \mathbf{W}\mathbf{h}_u\right).$$

The query term $\mathbf{a}_1^\top \mathbf{W}\mathbf{h}_v$ is the same constant for all candidates u, and LeakyReLU is monotonic, so the ranking of neighbours is decided by $\mathbf{a}_2^\top \mathbf{W}\mathbf{h}_u$ alone. Every query node ranks the keys in the same order (the same "best" neighbour for everyone) within one attention head.

GATv2 fix. Apply the non-linearity *before* the attention vector:

$$e_{vu} = \mathbf{a}^\top \mathrm{LeakyReLU}\left(\mathbf{W}[\mathbf{h}_v \,\|\, \mathbf{h}_u]\right).$$

The query and key features now interact non-linearly before the final score is computed, so the score is no longer a monotonic function of a key-only term. This gives *dynamic* attention: the neighbour ranking can depend on the query node.

Why it helps. Different nodes can prioritise different neighbours according to their own features and context. This makes neighbour selection more flexible and lets GATv2 capture query-dependent relationships that the original GAT cannot represent, at the same parameter cost.

Question 11: (Attentive GNNs) 5 points

(a) **[2 pts]**

$\exp(1.4) \approx 4$ and $\exp(0.7) \approx 2$, so

$$\beta_{\Phi_1} = \tfrac{4}{6} = \mathbf{\tfrac{2}{3}}, \qquad \beta_{\Phi_2} = \tfrac{2}{6} = \mathbf{\tfrac{1}{3}},$$

$$\mathbf{z} = \tfrac{2}{3}(3,0,6) + \tfrac{1}{3}(0,3,3) = (2,0,4) + (0,1,1) = \boxed{(2,1,5)}.$$

The model trusts **PAP** (shared authors) twice as much as PVP (same venue). Sharing an author is a more specific signal than sharing a venue, which contains many unrelated papers.

(b) **[3 pts]**

Node-level attention: for one meta-path Φ, learns which meta-path neighbours matter most to node i (α_{ij}^{Φ}, e.g. which co-authored papers).

Semantic-level attention: learns how much each meta-path (relation type) matters for the task overall (β_{Φ}, as in (a)).

Why both: importance exists at two scales. Individual neighbours differ in usefulness within a relation, and whole relations differ in usefulness for the task.

— End of Answer Key —